Experiment/Practical 7 Logistic regression

Title: Implementation of Logistic Regression for Classification

Aim: To implement and understand the Logistic Regression algorithm for classification tasks and evaluate its performance on a given dataset.

Objective: Students will learn

- Understand the logistic regression classification mechanism.
- Implement the logistic regression algorithm on a dataset for classification.
- Evaluate the performance using relevant metrics such as accuracy, precision, recall, and F1-score.
- Visualize the decision boundary of the logistic regression model.

Problem statement

Use the given dataset(s) to demonstrate the application of the Logistic Regression algorithm for classification. The task is to classify the data points into different classes based on the features and to understand how logistic regression models the probability of class membership using a sigmoid function.

Explanation/Stepwise Procedure/ Algorithm:

- Give a brief description of the logistic regression.

  Logistic regression is a data analysis technique that uses mathematics to find the relationships between two data factors. It then uses this relationship to predict the value of one of those factors based on the other. The prediction usually has a finite number of outcomes, like yes or no.

  Logistic regression is an important technique in the field of artificial intelligence and machine learning (AI/ML). ML models are software programs that you can train to perform complex data processing tasks without human intervention. ML models built using logistic regression help organizations gain actionable insights from their business data. They can use these insights for predictive analysis to reduce operational costs, increase efficiency, and scale faster. For example, businesses can uncover patterns that improve employee retention or lead to more profitable product design.

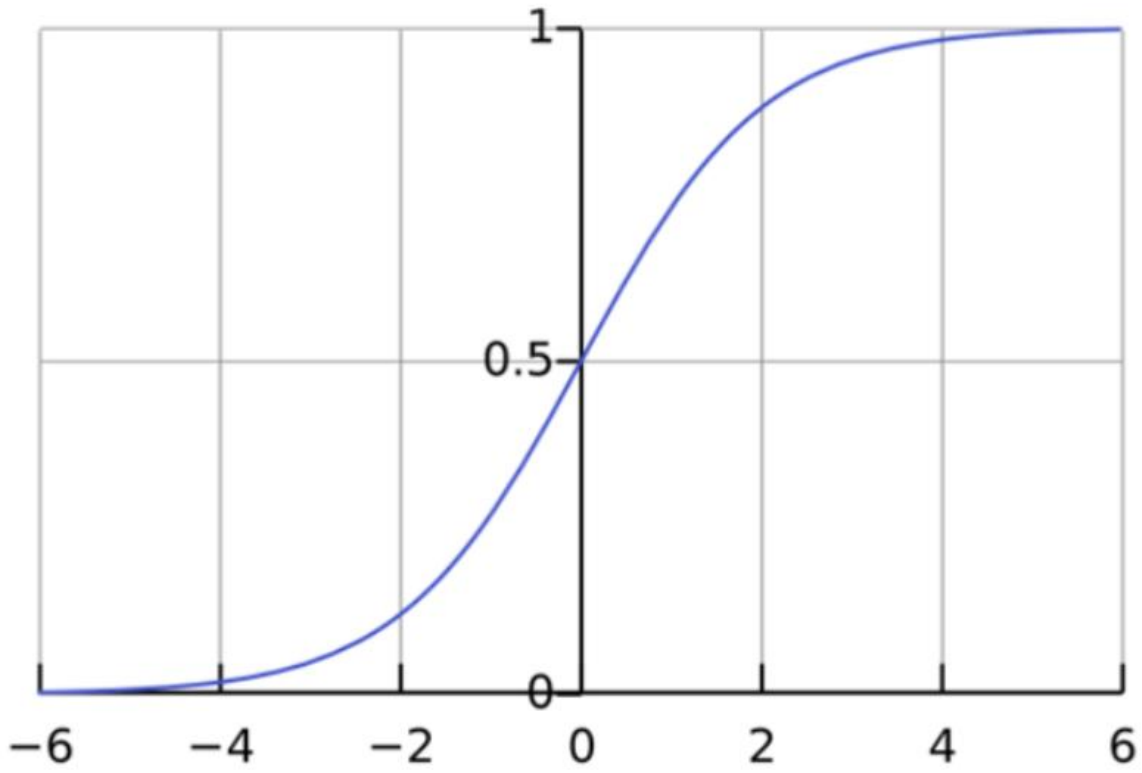
- Give mathematical formulation of logistic regression, including the sigmoid function and loss function (log loss/cross-entropy loss).

Logistic regression function

Logistic regression is a statistical model that uses the logistic function, or logit function, in mathematics as the equation between *x* and *y*. The logit function maps *y* as a sigmoid function of *x*.

$$f(x) = \frac{1}{1 + e^{-x}}$$

If you plot this logistic regression equation, you will get an S-curve as shown below.



The log function returns only values between 0 and 1 for the dependent variable, irrespective of the values of the independent variable. This is how logistic regression estimates the value of the dependent variable. Logistic regression methods also model equations between multiple independent variables and one dependent variable.

- Write the importance of the logistic regression in the classification task

- **Simple & Interpretable** – Easy to understand and implement.
- **Probability-Based** – Provides probabilities for class predictions.
- **Handles Binary & Multi-Class Classification** – Supports both types efficiently.
- **Less Prone to Overfitting** – Works well with regularization.
- **Feature Selection** – Coefficients indicate feature importance.
- **Works with Large Datasets** – Efficient for big data classification tasks.

- Mention applications of the logistic regression.

- **Healthcare** – Used in disease prediction models, such as detecting diabetes or heart disease.
- **Finance** – Helps in credit risk assessment and loan default prediction.
- **Email Filtering** – Identifies spam emails from genuine ones.
- **Customer Retention** – Predicts customer churn for businesses.
- **Marketing** – Determines the probability of a customer making a purchase.
- **Fraud Prevention** – Detects suspicious transactions in banking and e-commerce.
- **Social Media Insights** – Analyzes sentiment in user comments and reviews.

- Brief explanation of performance metrics (e.g., accuracy, precision, recall, f1-score, confusion matrix, cross-validation)

Performance Metrics in Machine Learning

1. Accuracy
   Accuracy measures the proportion of correctly classified instances out of the total instances. It is useful when the dataset is balanced but can be misleading if the classes are imbalanced.

   Formula:

$$\text{Accuracy} = \frac{(TP + TN)}{(TP + FP + TN + FN)}$$

2. Precision
   Precision evaluates how many of the predicted positive instances were actually positive. It is important in scenarios where false positives are costly.

   Formula:

$$Precision = \frac{TP}{TP + FP}$$

3. Recall (Sensitivity or True Positive Rate)
   Recall measures how many actual positive instances were correctly predicted. It is crucial when missing a positive instance is costly, such as in medical diagnoses.

   Formula:

$$Recall = \frac{TP}{TP + FN}$$

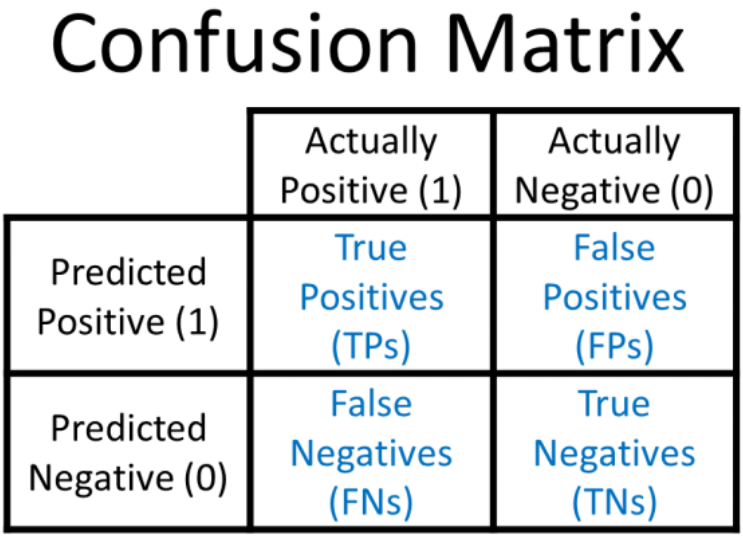
4. F1-Score
   The F1-score is the harmonic mean of precision and recall, providing a balanced measure when there is an uneven class distribution.

   Formula:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

5. Confusion Matrix
   A confusion matrix is a table used to evaluate the performance of a classification model by showing the counts of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN).

Actual Positive - Predicted Positive (TP), Predicted Negative (FN)
Actual Negative - Predicted Positive (FP), Predicted Negative (TN)



6. Cross-Validation
   Cross-validation is a technique for assessing the performance of a model by splitting the dataset into multiple training and testing subsets. The most common method is k-fold cross-validation, where the dataset is divided into k subsets, and the model is trained k times, each time using a different subset as the test set.

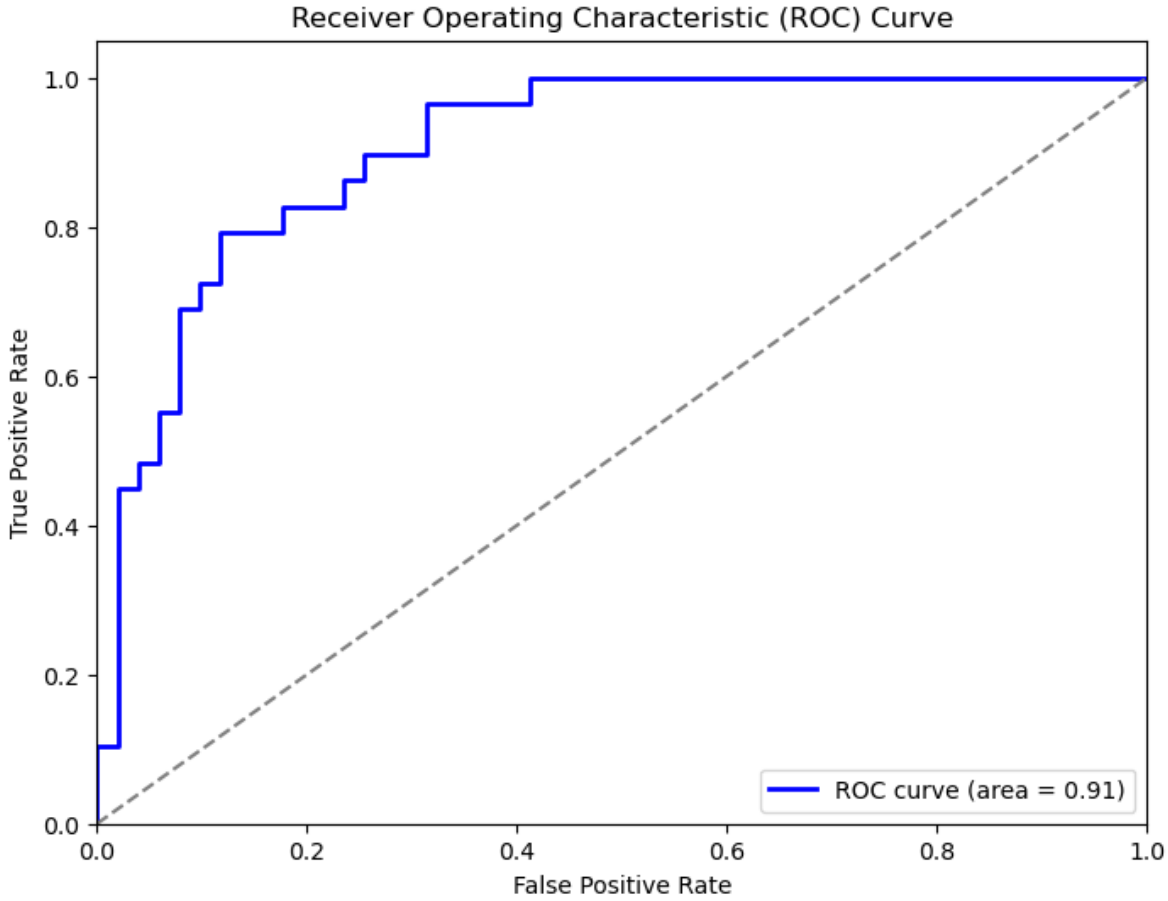
   This technique helps in reducing overfitting and provides a more reliable measure of a model's generalization ability.

Input & Output:

Input: Practice dataset Social_Network_Ads

- Dataset: Practice dataset Social_Network_Ads
- User Input: None
- Output: Purchase Prediction
- Predictions: Purchased
- Model Evaluation Metrics: Accuracy = 0.8125 , precision = 0.705 , recall = 0.620, F1-score = 0.705, confusion matrix = [[47 4] [11 18]] etc.
- Visualizations: Plots showing decision boundaries and performance evaluation graphs

- ***Add necessary figure(s)/Diagram(s)***



Conclusion:

The logistic regression model successfully classified the given dataset with decent accuracy. Data preprocessing steps like encoding categorical features and scaling improved model performance. The results indicate that logistic regression is effective for binary classification tasks, but further optimization and feature engineering could enhance its predictive power.

SML_Logistic_LAB_ASSIGNMENT

March 13, 2025

[113]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, accuracy_score,␣
 ↪classification_report, f1_score, precision_score, recall_score,␣
 ↪roc_auc_score, roc_curve, auc, accuracy_score, confusion_matrix,␣
 ↪classification_report, auc, precision_recall_curve
from imblearn.over_sampling import SMOTE
import seaborn as sns
from sklearn.linear_model import LogisticRegression
```

[114]:
```
url = r"D:\Sem_4\Supervised Machine Learning lab (SMLL)\8\Practice dataset␣
 ↪Social_Network_Ads.csv"
df = pd.read_csv(url)
df.head()
```

[114]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

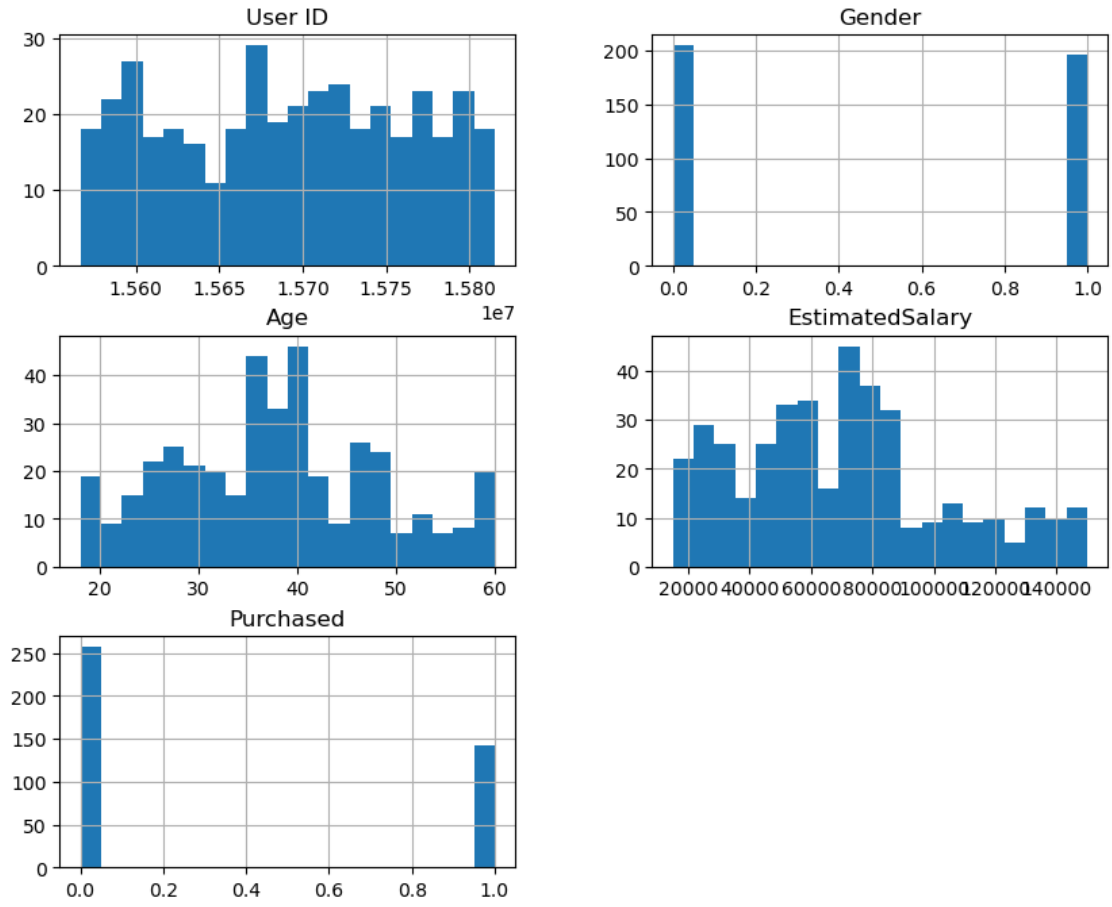
[115]:
```
# Display basic information about the dataset
print(f"The dataset has {df.shape[0]} rows and {df.shape[1]} columns.\n")
print(f"The dataset has {df.isnull().sum().sum()} missing values.\n")
print(f"The dataset has {df.duplicated().sum()} duplicated rows.\n")
print(f"The dataset has columns\n", df.columns)
print(f"The dataset has description\n", df.describe())
```

```
The dataset has 400 rows and 5 columns.

The dataset has 0 missing values.

The dataset has 0 duplicated rows.
```

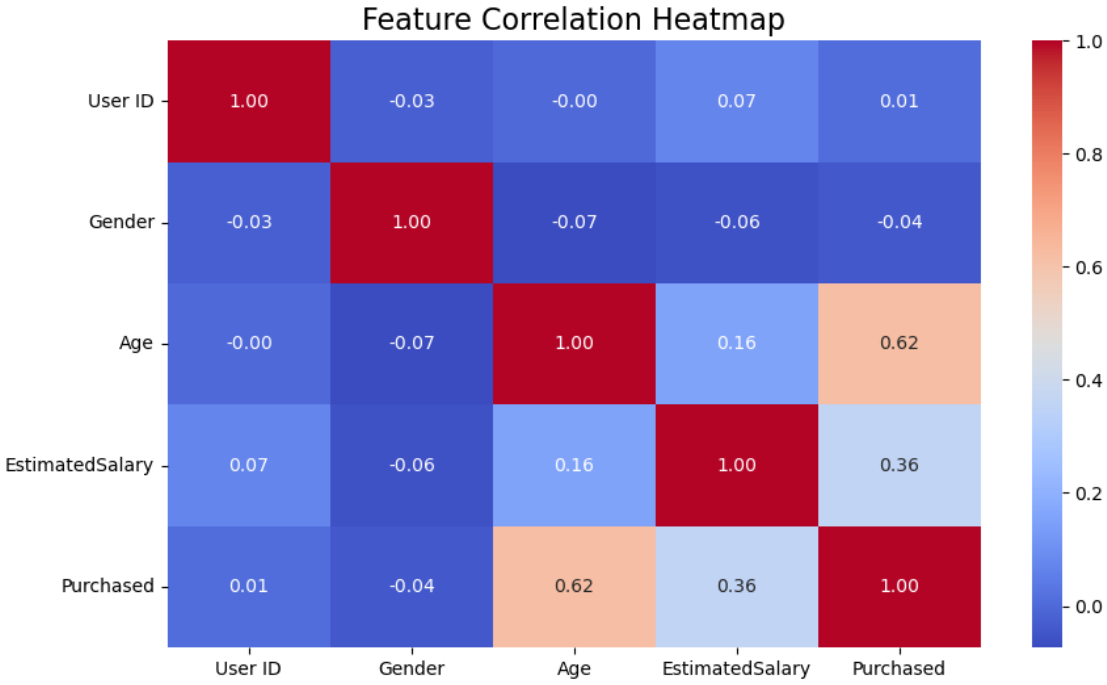
```
The dataset has columns
 Index(['User ID', 'Gender', 'Age', 'EstimatedSalary', 'Purchased'],
dtype='object')
The dataset has description
             User ID         Age  EstimatedSalary   Purchased
count  4.000000e+02  400.000000       400.000000  400.000000
mean   1.569154e+07   37.655000     69742.500000    0.357500
std    7.165832e+04   10.482877     34096.960282    0.479864
min    1.556669e+07   18.000000     15000.000000    0.000000
25%    1.562676e+07   29.750000     43000.000000    0.000000
50%    1.569434e+07   37.000000     70000.000000    0.000000
75%    1.575036e+07   46.000000     88000.000000    1.000000
max    1.581524e+07   60.000000    150000.000000    1.000000
```

[116]:
```
# Data Preprocessing
# Encoding categorical data
label_encoder = LabelEncoder()
df["Gender"] = label_encoder.fit_transform(df["Gender"])
```

[117]:
```
# Data Distribution
plt.Figure(figsize=(10, 5))
df.hist(figsize=(10, 8), bins=20)
plt.suptitle("Data Distribution", fontsize=16)
plt.show()
```



[118]:
```
# Correlation heatmap
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Feature Correlation Heatmap", fontsize=16)
plt.show()
```



[119]:
```
# Splitting Data into Features and Target
X = df.drop("Purchased", axis=1)
y = df["Purchased"]
print(X.shape, y.shape)
```

```
(400, 4) (400,)
```

[120]:
```
# Splitting Data into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42, stratify=y)
```

[121]:
```
# Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

[122]:
```
# Model Training - Logistic Regression
log_reg = LogisticRegression()
log_reg.fit(X_train_scaled, y_train)
```

[122]:
```
LogisticRegression()
```

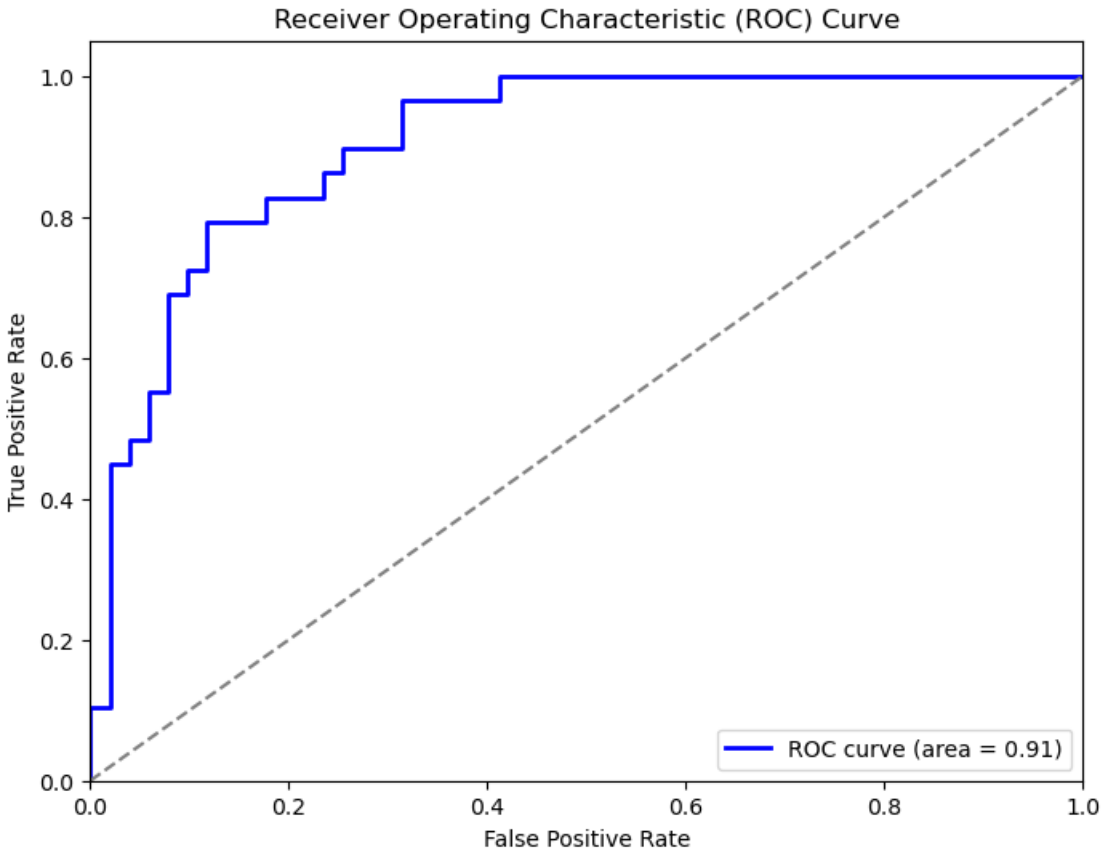
[123]:
```
# Model Prediction
y_pred = log_reg.predict(X_test_scaled)
y_pred_proba = log_reg.predict_proba(X_test_scaled)[:, 1]
```

[124]:
```
# Model Evaluation
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred_proba)
print("Accuracy:", accuracy)
print("F1 Score:", f1)
print("Precision:", precision)
print("Recall:", recall)
print("ROC AUC Score:", roc_auc)
print("classification_report:\n", classification_report(y_test, y_pred))
print("confusion_matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.8125
F1 Score: 0.7058823529411765
Precision: 0.8181818181818182
Recall: 0.6206896551724138
ROC AUC Score: 0.9066937119675457
classification_report:
              precision    recall  f1-score   support

           0       0.81      0.92      0.86        51
           1       0.82      0.62      0.71        29

    accuracy                           0.81        80
   macro avg       0.81      0.77      0.78        80
weighted avg       0.81      0.81      0.81        80

confusion_matrix:
 [[47  4]
 [11 18]]
```

[125]:
```
# ROC - AUC curve
fpr,tpr,_ = roc_curve(y_test, y_pred_proba)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8,6))
plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC curve (area = {roc_auc:.
 ↪2f})')
plt.plot([0, 1], [0, 1], color='grey', linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
```

```
plt.show()
```


Receiver Operating Characteristic (ROC) Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC curve (area = 0.91)